

Министерство Образования Российской Федерации
Пермский Государственный Национальный Исследовательский
университет
Государственного и муниципального управления

ОТЧЁТ
по лабораторной работе №1
по дисциплине:
Язык программирования C++

Выполнил: студент гр. ИТ-7, 2 курс
Новиков Артём Денисович
Руководитель: Ракина В.Д.

г. Пермь, 2026

Задание 1

Задача 2. Сумма знаков

Текст задачи

Сигнатура функции:

```
int sumLastNums (int x);
```

Необходимо реализовать функцию таким образом, чтобы она возвращала результат сложения двух последних знаков числа, предполагая, что знаков в числе не менее двух.

Подсказки:

```
int x = 123 % 10; // x будет иметь значение 3
```

```
int y = 123 / 10; // y будет иметь значение 12
```

Пример: x = 4568, результат: 14

Алгоритм решения

Пользователь вводит число x (в нём должно быть минимум две цифры). Последнюю цифру получаем так: $x \% 10$. Предпоследнюю так: $(x \% 100) / 10$. Складываем две цифры и выводим результат.

Тестирование

```
Task 1.2
Enter number x:4568

Sum two last nums: 14
```

```
Task 1.2
Enter number x:-12

Invalid input
```

```
Task 1.2
Enter number x:qwert

Invalid input
```

Задача 4. Есть ли позитив

Текст задачи

Сигнатура функции:

```
bool isPositive (int x);
```

Необходимо реализовать функцию таким образом, чтобы она принимала число x и возвращала `true`, если оно положительное.

Пример 1: $x = 3$, результат: `true`

Пример 2: $x = -5$, результат: `false`

Алгоритм решения

Пользователь вводит число x . Если x больше нуля, функция возвращает `true`, иначе `false`. Результат выводится на экран.

Тестирование

```
Task 1.4
Enter number x:3
1
```

```
Task 1.4
Enter number x:-3
0
```

```
Task 1.4
Enter number x:qwe
Invalid input
```

Задача 6. Большая буква

Текст задачи

Сигнатура функции:

```
bool isUpperCase (char x);
```

Необходимо реализовать функцию таким образом, чтобы она принимала символ `x` и возвращала `true`, если это большая буква в диапазоне от 'A' до 'Z'.

Пример 1: `x = 'D'`, результат: `true`

Пример 2: `x = 'q'`, результат: `false`

Алгоритм решения

Пользователь вводит символ `x`. Если символ находится между 'A' и 'Z', функция возвращает `true`, иначе `false`. Если введена цифра, программа пишет, что ввод неверный.

Тестирование

```
Task 1.6
Enter symbol x:D
1
```

```
Task 1.6
Enter symbol x:d
0
```

```
Task 1.6
Enter symbol x:1

Invalid input
```

Задача 8. Делитель

Текст задачи

Сигнатура функции:

```
bool isDivisor (int a, int b);
```

Необходимо реализовать функцию таким образом, чтобы она возвращала true, если любое из принятых чисел делит другое нацело.

Пример 1: $a = 3$, $b = 6$, результат: true

Пример 2: $a = 2$, $b = 15$, результат: false

Алгоритм решения

Пользователь вводит два числа a и b (ни одно не должно быть нулём). Если a делится на b без остатка или b делится на a без остатка, функция возвращает true, иначе false. Остаток считаем через %.

Тестирование

```
Task 1.8
Enter number a:a
Invalid input
```

```
Task 1.8
Enter number a:3
Enter number b:6
1
```

```
Task 1.8
Enter number a:2
Enter number b:15
0
```

Задача 10. Многократный вызов

Текст задачи

Сигнатура функции:

```
int lastNumSum (int a, int b);
```

Необходимо реализовать функцию таким образом, чтобы она считала сумму цифр двух чисел из разряда единиц. Выполните с его помощью последовательное сложение пяти чисел и результат выведите на экран. Постарайтесь выполнить задачу, используя минимально возможное количество вспомогательных переменных.

Пример: $5 + 11$ это 6

$6 + 123$ это 9

$9 + 14$ это 13

$13 + 1$ это 4

Итого: 4

Алгоритм решения

Пользователь вводит первое число и запоминает его в result. Затем он по очереди вводит ещё четыре числа. Каждый раз функция берёт последнюю цифру у result и у нового числа (через $\% 10$), складывает их, и сумма снова записывается в result. После последнего числа выводится итог.

Тестирование

```
Task 1.10
Enter number a:5

Enter number b:11

Current sum: 6
Enter next number:123

Current sum: 9
Enter next number:14

Current sum: 13
Enter next number:1

Final result: 4
```

```
Task 1.10
Enter number a:-11

Enter number b:-123

Current sum: -4
Enter next number:-512

Current sum: -6
Enter next number:-13

Current sum: -9
Enter next number:-98

Final result: -17
```

```
Task 1.10
Enter number a:qwe

Invalid input
```

Задание 2

Задача 2. Безопасное деление

Текст задачи

Сигнатура функции:

```
double safeDiv (int x, int y);
```

Необходимо реализовать функцию таким образом, чтобы она возвращала деление x на y , и при этом гарантировала, что не будет выкинута ошибка деления на 0. При делении на 0 следует вернуть из функции число 0.

Подсказка: смотри ограничения на операции типов данных.

Пример 1: $x = 5$, $y = 0$, результат: 0

Пример 2: $x = 8$, $y = 2$, результат: 4

Алгоритм решения

Пользователь вводит x и y . Если y равно 0, функция возвращает 0, чтобы не делить на ноль. Иначе x превращается в `double` и делится на y . Результат выводится на экран.

Тестирование

```
Task 2.2
Enter number x:5
Enter number y:0
Result: 0
```

```
Task 2.2
Enter number x:8
Enter number y:4
Result: 2
```

```
Task 2.2
Enter number x:-4
Enter number y:-20
Result: 0.2
```

```
Task 2.2
Enter number x:a
Invalid input
```

Задача 4. Строка сравнения

Текст задачи

Сигнатура функции:

String makeDecision (int x, int y);

Необходимо реализовать функцию таким образом, чтобы она возвращала строку, которая включает два принятых функцией числа и корректно выставленный знак операции сравнения (больше, меньше или равно).

Пример 1: $x = 5, y = 7$, результат: "5<7"

Пример 2: $x = 8, y = -1$, результат: "8>-1"

Пример 3: $x = 4, y = 4$, результат: "4==4"

Алгоритм решения

Пользователь вводит x и y . Функция сравнивает числа и возвращает строку: если x больше, то " $x > y$ ", если меньше, то " $x < y$ ", если равны, то " $x == y$ ". Числа в строку превращаем через `std::to_string`.

Тестирование

```
Task 2.4
Enter number a:5
Enter number b:7
5<7
```

```
Task 2.4
Enter number a:2
Enter number b:2
2==2
```

```
Task 2.4
Enter number a:7
Enter number b:5
7>5
```

```
Task 2.4
Enter number a:qew
Invalid input
```

Задача 6. Тройная сумма

Текст задачи

Сигнатура функции:

```
bool sum3 (int x, int y, int z);
```

Необходимо реализовать функцию таким образом, чтобы она возвращала true, если два любых числа (из трёх принятых) можно сложить так, чтобы получить третье.

Пример 1: $x = 5$, $y = 7$, $z = 2$, результат: true

Пример 2: $x = 8$, $y = -1$, $z = 4$, результат: false

Алгоритм решения

Пользователь вводит три числа x , y , z . Функция проверяет три случая: x равно $y+z$, y равно $z+x$, z равно $x+y$. Если выполняется хотя бы один, возвращается true, иначе false.

Тестирование

```
Task 2.6
Enter number a:5
Enter number b:7
Enter number c:2
1
```

```
Task 2.6
Enter number a:8
Enter number b:-1
Enter number c:4
0
```

```
Task 2.6
Enter number a:0
Enter number b:a
Invalid input
```

Задача 8. Возраст

Текст задачи

Сигнатура функции:

String age (int x);

Необходимо реализовать функцию таким образом, чтобы она возвращала строку, в которой сначала будет число x , а затем одно из слов: год, года, лет.

Слово “год” добавляется, если число x заканчивается на 1, кроме числа 11. Слово “года” добавляется, если число x заканчивается на 2, 3 или 4, кроме чисел 12, 13, 14. Слово “лет” добавляется во всех остальных случаях.

Подсказка: оператор % позволяет получить остаток от деления.

Пример 1: $x = 5$, результат: “5 лет”

Пример 2: $x = 31$, результат: “31 год”

Пример 3: $x = 44$, результат: “44 года”

Алгоритм решения

Пользователь вводит возраст x (не меньше 0). Если число оканчивается на 1, но это не 11, добавляем “год”. Если оканчивается на 2, 3 или 4, но это не 12, 13, 14, добавляем “года”. Во всех остальных случаях добавляем “лет”. Последнюю цифру берём через $x \% 10$.

Тестирование

Примечание: В английском языке слова “год, года, лет” переводятся одним словом *year*, поэтому в программе пришлось обозначать: год – *god*, года – *goda*, лет – *let*, также русский язык в программе при выводе пользователю выводит случайные символы, которые невозможно прочесть.

```
Task 2.8
Enter number x:5

5 let
```

```
Task 2.8
Enter number x:31

31 god
```

```
Task 2.8
Enter number x:44

44 goda
```

```
Task 2.8
Enter number x:qwe

Invalid input
```

Задача 10. Вывод дней недели

Текст задачи

Сигнатура функции:

```
void printDays (int x);
```

В качестве параметра функция принимает число x , обозначающее день недели. Необходимо реализовать функцию таким образом, чтобы она выводила на экран название переданного дня и всех последующих до конца недели дней. Если в качестве параметра передан не день (число не в диапазоне от 1 до 7), то выводится текст “это не день недели”. Первый день – понедельник, последний – воскресенье. Вместо `if` в данной задаче используйте `switch`.

Пример 1: $x = 4$, результат: четверг, пятница, суббота, воскресенье

Пример 2: $x = 18$, результат: это не день недели

Алгоритм решения

Пользователь вводит число x . В `switch` для каждого дня недели от 1 до 7 написан свой `case`. `Break` стоит только в конце, после воскресенья. Поэтому программа выводит выбранный день и все следующие за ним до воскресенья. Если число не от 1 до 7, срабатывает `default` и выводится сообщение, что это не день недели.

Тестирование

```
Task 2.10
Enter number x:4

Thursday
Friday
Saturday
Sunday
```

```
Task 2.10
Enter number x:13

Not day week
```

```
Task 2.10
Enter number x:qwer

Invalid input
```

Задание 3

Задача 2. Числа наоборот

Текст задачи

Сигнатура функции:

```
String reverseListNums (int x);
```

Необходимо реализовать функцию таким образом, чтобы она возвращала строку, в которой будут записаны все числа от x до 0 (включительно).

Пример: $x = 5$, результат: “5 4 3 2 1 0”

Алгоритм решения

Пользователь вводит число x (не меньше 0). Создаём пустую строку result. Цикл идёт от x до 0 и на каждом шаге добавляет в строку текущее число и пробел. После цикла строка возвращается и выводится.

Тестирование

```
Task 3.2
Enter a number:5

5 4 3 2 1 0
```

```
Task 3.2
Enter a number:qwer

Invalid input
```

```
Task 3.2
Enter a number:-12

Invalid input
```

Задача 4. Степень числа

Текст задачи

Сигнатура функции:

```
int pow (int x, int y);
```

Необходимо реализовать функцию таким образом, чтобы она возвращала результат возведения x в степень y .

Подсказка: для получения степени необходимо умножить единицу на число x и сделать это y раз, т.е. два в третьей степени это $1*2*2*2$.

Пример: $x = 2$, $y = 5$, результат: 32

Алгоритм решения

Пользователь вводит x и y . Создаём $result = 1$. Цикл повторяется y раз, и на каждом шаге $result$ умножается на x . После цикла $result$ и есть x в степени y .

Тестирование

```
Task 3.4
Enter a number x:2
Enter a number y:5
32
```

```
Task 3.4
Enter a number x:-1
Enter a number y:3
-1
```

```
Task 3.4
Enter a number x:qwer
Invalid input
```

Задача 6. Одинаковость

Текст задачи

Сигнатура функции:

```
bool equalNum (int x);
```

Необходимо реализовать функцию таким образом, чтобы она возвращала true, если все знаки числа одинаковы, и false в ином случае.

Подсказки:

```
int x = 123 % 10; // x будет иметь значение 3
```

```
int y = 123 / 10; // y будет иметь значение 12
```

Пример 1: x = 1111, результат: true

Пример 2: x = 1211, результат: false

Алгоритм решения

Пользователь вводит число x. Если оно отрицательное, делаем его положительным. Запоминаем последнюю цифру (x % 10) и отбрасываем её (x / 10). Пока в числе остались цифры, берём следующую и сравниваем с запомненной. Если нашлась другая, возвращаем false. Если все цифры совпали, возвращаем true.

Тестирование

```
Task 3.6
Enter a number x:1111

1
```

```
Task 3.6
Enter a number x:qwer

Invalid input
```

```
Task 3.6
Enter a number x:1211

0
```

Задача 8. Левый треугольник

Текст задачи

Сигнатура функции:

```
void leftTriangle (int x);
```

Необходимо реализовать функцию таким образом, чтобы она выводила на экран треугольник из символов '*', у которого x символов в высоту, а количество символов в ряду совпадает с номером строки.

Пример 1: $x = 2$, результат:

```
*  
**
```

Пример 2: $x = 4$, результат:

```
*  
**  
***  
****
```

Алгоритм решения

Пользователь вводит число x . Внешний цикл идёт по строкам от 1 до x . Внутренний цикл в каждой строке печатает столько звёздочек, каков номер строки. После строки переходим на новую.

Тестирование

```
Task 3.8  
Enter number x:4  
  
*  
**  
***  
****
```

```
Task 3.8  
Enter number x:-1  
  
Invalid input
```

Задача 10. Угадайка

Текст задачи

Сигнатура функции:

```
void guessGame ();
```

Необходимо реализовать функцию таким образом, чтобы она генерировала случайное число от 0 до 9, далее считывала с консоли введенное пользователем число и выводила, угадал ли пользователь то, что было загадано, или нет. Функция запускается до тех пор, пока пользователь не угадает число. После этого выведите на экран количество попыток, которое потребовалось пользователю, чтобы угадать число.

Пример:

Введите число от 0 до 9: 5

Вы не угадали,

введите число от 0 до 9: 9

Вы угадали! Вы отгадали число за 2 попытки

Алгоритм решения

Программа загадывает случайное число от 0 до 9 и ставит счётчик попыток в 0. В цикле пользователь вводит число, и счётчик увеличивается на 1. Если число совпало с загаданным, выводим “You win” и количество попыток, и цикл заканчивается. Если нет, выводим “You lose” и просим ввести число снова.

Тестирование

```
Task 3.10
Enter number from 0 to 9:8

You lose
Enter number from 0 to 9:4

You lose
Enter number from 0 to 9:231

Invalid input
```

```
Task 3.10
Enter number from 0 to 9:1

You lose
Enter number from 0 to 9:2

You lose
Enter number from 0 to 9:3

You lose
Enter number from 0 to 9:4

You win
You guessed the number in 4 attempts
```

Задание 4

Задача 2. Поиск последнего значения

Текст задачи

Сигнатура функции:

```
int findLast (int arr[], int x);
```

Необходимо реализовать функцию таким образом, чтобы она возвращала индекс последнего вхождения числа x в массив arr . Если число не входит в массив – возвращается -1.

Пример: $arr = [1, 2, 3, 4, 2, 2, 5]$, $x = 2$, результат: 5

Алгоритм решения

Пользователь вводит 7 чисел массива и число x . Создаём $lastIndex = -1$. Цикл проходит по всем элементам, и если элемент равен x , запоминаем его индекс в $lastIndex$. После цикла в $lastIndex$ лежит индекс последнего совпадения, а если совпадений не было, остаётся -1.

Тестирование

```
Task 4.2
Enter 7 elements of array:1 2 3 4 2 2 5

Enter number to find:2

Last index: 5
```

```
Task 4.2
Enter 7 elements of array:1 2 3 2 2 4 5

Enter number to find:5

Last index: 6
```

```
Task 4.2
Enter 7 elements of array:1 2 3 4 5 6 7

Enter number to find:123

Last index: -1
```


Задача 4. Добавление в массив

Текст задачи

Сигнатура функции:

```
int* add (int arr[], int x, int pos);
```

Необходимо реализовать функцию таким образом, чтобы она возвращала новый массив, который будет содержать все элементы массива `arr`, однако в позицию `pos` будет вставлено значение `x`.

Пример: `arr = [1, 2, 3, 4, 5]`, `x = 9`, `pos = 3`, результат: `[1, 2, 3, 9, 4, 5]`

Алгоритм решения

Пользователь вводит 5 чисел, число `x` и позицию `pos` (от 0 до 5). Создаём новый массив на 6 элементов. Копируем в него элементы исходного массива до позиции `pos`, на позицию `pos` ставим `x`. Остальные элементы копируем дальше, со сдвигом на одну клетку. Новый массив выводим и потом освобождаем память через `delete[]`.

Тестирование

```
Task 4.4
Enter 5 elements of array:1 2 3 4 5

Enter number to insert:9

Enter position (0-5):3

New array: 1 2 3 9 4 5
```

```
Task 4.4
Enter 5 elements of array:1 2 3 4 5

Enter number to insert:9

Enter position (0-5):-1

Error, Position must be from 0 to 5
```

```
Task 4.4
Enter 5 elements of array:1

2

2

3

3

Enter number to insert:123

Enter position (0-5):йцуй

Error, Enter a number only
```

Задача 6. Реверс

Текст задачи

Сигнатура функции:

```
void reverse (int arr[]);
```

Необходимо реализовать функцию таким образом, чтобы она изменяла массив arr. После проведённых изменений массив должен быть записан задом наперёд.

Пример: arr = [1, 2, 3, 4, 5], результат: arr = [5, 4, 3, 2, 1]

Алгоритм решения

Пользователь вводит 5 чисел. Функция проходит только по первой половине массива и меняет местами симметричные элементы: первый с последним, второй с предпоследним. Для обмена используется переменная temp. После этого массив выводим.

Тестирование

```
Task 4.6
Enter 5 elements of array:1 2 3 4 5
Reversed array: 5 4 3 2 1
```

```
Task 4.6
Enter 5 elements of array:qwe
Error, Enter numbers only
```

Задача 8. Объединение

Текст задачи

Сигнатура функции:

```
int* concat (int arr1[], int arr2[]);
```

Необходимо реализовать функцию таким образом, чтобы она возвращала новый массив, в котором сначала идут элементы первого массива (arr1), а затем второго (arr2).

Пример: arr1 = [1, 2, 3], arr2 = [7, 8, 9], результат: [1, 2, 3, 7, 8, 9]

Алгоритм решения

Пользователь вводит два массива по 3 числа. Создаём новый массив на 6 элементов. Сначала копируем в него первый массив, потом второй, начиная с позиции 3. Новый массив выводим и освобождаем память.

Тестирование

```
Task 4.8
Enter 3 elements of first array:1 2 3

Enter 3 elements of second array:4 5 6

Combined array: 1 2 3 4 5 6
```

```
Task 4.8
Enter 3 elements of first array:qwe

Error, Enter numbers only
```

Задача 10. Удалить негатив

Текст задачи

Сигнатура функции:

```
int* deleteNegative (int arr[]);
```

Необходимо реализовать функцию таким образом, чтобы она возвращала новый массив, в котором записаны все элементы массива arr, кроме отрицательных.

Пример: arr = [1, 2, -3, 4, -2, 2, -5], результат: [1, 2, 4, 2]

Алгоритм решения

Пользователь вводит 7 чисел. Сначала считаем, сколько в массиве неотрицательных чисел, и создаём новый массив такого размера. Затем проходим по исходному массиву, и каждое неотрицательное число записываем в новый массив через отдельный индекс j (после записи j увеличивается на 1). Новый массив выводим и освобождаем память.

Тестирование

```
Task 4.10
Enter 7 numbers:1 2 -2 3 -4 4 5
Result: 1 2 3 4 5
```

```
Task 4.10
Enter 7 numbers:qew
Error, Enter numbers only
```